

Security Analysis of ML-Powered Android Apps: A Static-Analysis Pipeline for On-Device ML Vulnerability Detection

Bruna Vasconcelos
School of Computing
University of Utah
Salt Lake City, UT, USA
bruna.gvasconcelos54@gmail.com

Abstract—On-device machine learning is increasingly embedded in mobile applications, yet standard Android security tooling (FlowDroid, TaintDroid, Soot) predates widespread on-device ML and carries no awareness of ML-specific artifacts, SDKs, or data flows. We present a static-analysis pipeline that augments APK analysis with ML-aware detection: it identifies embedded model files via extension and magic-byte matching, quantifies their exposure with Shannon entropy, flags sensitive permissions mapped to ML data sources, and scans decompiled Smali bytecode for ML-SDK imports and poisoning-surface signatures. Detection outputs are scored against a Critical / High / Medium / Info rubric. We apply the pipeline to a pilot corpus of five F-Droid applications (274MB) and a synthetic positive-control APK containing a public MobileNet v1 model. The positive control is correctly identified at High severity; none of the five real apps bundle on-device ML. We treat the null result as a pilot observation rather than a structural claim about F-Droid, since a five-app sample covers $\approx 0.1\%$ of the catalog. The tool is architected to transfer directly to a Play Store corpus, where on-device ML is known to be more prevalent; the main engineering change required is raising the Smali scan cap and adding an obfuscation-resistant detector.

Index Terms—Android security, on-device machine learning, static analysis, data poisoning, membership inference, Truth Serum, APK analysis

I. INTRODUCTION

The *Truth Serum* attack of Tramèr et al. [1] amplifies membership inference against a model by inserting mislabeled near-duplicates of target samples into the training set. After training, the poisoned target samples exhibit elevated loss, making them trivially identifiable to an adversary. The attack presumes that an adversary can contribute training data – a presumption that maps onto any on-device ML pipeline that collects user-supplied data for retraining, personalization, or federated learning.

Existing mobile-ML security work measures orthogonal dimensions. Xu et al. [2] presented a first empirical study of deep learning apps on smartphones, characterising *presence* of ML across 16,500 Play Store apps. Sun et al. [3] released ModelXRay, a large-scale static analyser that quantifies *insufficient model protection*, showing that 41% of ML apps ship unprotected models and that 66% of protected models are still extractable via lightweight dynamic analysis. Standard

Android taint tools – FlowDroid [4] and TaintDroid [5] – predate widespread on-device ML and carry no ML-specific sources or sinks. None of these audits the *training-surface* precondition that enables Truth Serum: whether user-supplied data flows into on-device retraining or personalisation. That gap is the project’s target.

This paper makes three contributions:

- A static-analysis pipeline, implemented in under 500 lines of Python, that extends standard Android APK analysis with ML-aware detection.
- A severity rubric for on-device ML vulnerabilities with reproducible evidence (per-file entropy, magic-byte identification, and SHA-256 APK hashes).
- An empirical pilot on a five-app F-Droid corpus with a synthetic positive control, demonstrating that the detector fires as expected while documenting the dominant sources of false-negative risk (ProGuard/R8 obfuscation, Smali scan cap).

II. THREAT MODEL

Adversary. A malicious training-data contributor: any party that can influence data ingested by an app’s on-device training, personalization, or federated-learning loop. The adversary does not require code execution on the device; a poisoned input delivered through the normal data-collection channel is sufficient (the Truth Serum setting).

Defender. An app auditor with access to a publicly distributed APK and no access to the running app, backend services, or runtime instrumentation. Static analysis only.

Assets at risk. Model weights (integrity and confidentiality), the training-data pipeline (integrity), and user personally identifiable information (PII) routed through inference.

Out of scope. Runtime exploits, supply-chain attacks on the build pipeline, backend-service compromises, and app repackaging / Frida-based dynamic analysis – all deferred to future work under the one-day pilot scope.

III. RESEARCH QUESTIONS

Primary (minimum viable deliverable):

- **RQ1:** Do poisoning surfaces exist in distributed Android apps? Does user-submitted data flow into on-device re-training without sanitization or anomaly detection?
- **RQ2:** Are model files exposed? Are `.tflite`, `.onnx`, or similar artifacts bundled unencrypted inside APK assets with no integrity check on load?

Stretch:

- **RQ3:** Is training data handled securely (world-readable directories, unsanitized input)?
- **RQ4:** Are model update channels safe (TLS, signed updates)?

IV. METHODOLOGY

The analysis is static only. For each APK, the pipeline executes the following phases:

- 1) **SHA-256 hash** the APK bytes, supporting reproducibility.
- 2) **Archive scan.** Unzip without decoding. Match file extensions in `{.tflite,.onnx,.pb,.pt,.pkl,.h5,.mlmodel,.pth,.caffeemodel}`. For each hit, read the first 64 KB and compute Shannon entropy plus magic-byte identification. An entropy above 7.9 bits/byte is flagged as *high-entropy*: consistent with encryption *or* compression, which are indistinguishable from the bytes alone.
- 3) **Native-library scan.** Regex over archive entries for `libtensorflowlite*.so`, `libonnxruntime*.so`, `libmlkit*.so`, `libmediapipe*.so`, and `libpytorch*.so`.
- 4) **apktool decode**, with a 300s per-APK timeout to bound the impact of adversarially constructed APKs.
- 5) **Manifest parse.** `AndroidManifest.xml` is parsed with `defusedxml.ElementTree`, resisting XXE and entity-expansion attacks, rather than the naive regex approach often seen in similar pipelines. We extract the package name and flag a curated set of sensitive permissions mapped to ML data sources (CAMERA, RECORD_AUDIO, ACCESS_FINE_LOCATION, READ_CONTACTS, READ_SMS, READ/WRITE_EXTERNAL_STORAGE, INTERNET).
- 6) **Smali pattern scan.** Capped at the first 5000 Smali files per APK, sorted alphabetically so the scan is deterministic. A warning is emitted when the cap truncates the scan – silent truncation would mask false negatives. Two pattern families are applied: ML-SDK imports (`org.tensorflow/lite`, `com/google/mlkit`, `ai.onnxruntime`, `com/google/mediapipe`, `org.pytorch`) and poisoning-surface signatures (`modelPersonalizer / PersonalizationModel / OnDeviceCustomActions`, `federated-learning references`, `feedback-to-retraining phrases`, `GradientTape / keras.fit`, and `differentialPrivacy APIs`).

- 7) **Severity scoring.** Findings are scored Critical / High / Medium / Info by the rubric defined in the proposal.

The pipeline emits one JSON record per APK (including per-APK SHA-256 and both scanned and total Smali file counts) plus an aggregated CSV.

A. Scope cuts for the one-day pilot

The full methodology documented in the project proposal includes FlowDroid-based taint analysis and Frida dynamic validation. Both are deferred: FlowDroid’s configuration cost outweighs its yield on a one-day timeline, and its known reflection / dynamic-class-loading blind spots would require manual review regardless. Frida requires APK repackaging, estimated at approximately two days per batch. The pilot corpus is reduced from 30+ apps to five real apps plus one synthetic positive-control APK.

V. CORPUS AND ETHICS

The pilot corpus consists of five applications downloaded from F-Droid (open-source, freely redistributable, no CFAA concern) plus one synthetic positive-control APK assembled from a public MobileNet v1 `.tflite` file (Apache 2.0 license, Google Cloud Storage). Table I lists the corpus and SHA-256 prefixes for reproducibility.

TABLE I
PILOT CORPUS WITH SHA-256 PREFIXES

App	Size	SHA-256 (16)
F-Droid (<code>org.fdroid.fdroid</code>)	12.4 MB	985f5181d48bb6ba
NewPipe (<code>org.schabi.newpipe</code>)	10.9 MB	dbc8a1bb7a3db16f
Aves Gallery libre	55.9 MB	75697b19f2eb850f
Nextcloud Files	77.8 MB	e36e6ef4215cf003
Fennec (Firefox fork)	117.6 MB	251146a2b5f6d801
Positive control (synthetic)	15.7 MB	d9c71359fcce2d5f

All analysis is performed on freely redistributable APKs. No server interaction, no live user data, no repackaging, and no PoC delivery to real devices. Responsible disclosure is framed as a 90-day window should any critical finding emerge in future Play Store runs.

VI. FINDINGS

A. Pilot observation on the F-Droid corpus

None of the five F-Droid APKs bundle a model file, link an ML native library, or reference an ML SDK in the Smali that was scanned. Every app’s top finding is an Info-severity “No on-device ML artifacts detected.”

We treat this as a *pilot observation*, not a structural claim about F-Droid as a whole. Five apps is approximately 0.1% of the F-Droid catalog, and the Smali scan is capped (§VI-C); absence of evidence at this scale is not evidence of absence. The defensible reading is that F-Droid’s audit requirements *likely* filter out proprietary on-device ML, which should be confirmed on a larger corpus.

The actionable conclusion: F-Droid is not a productive corpus for this research question. The pipeline should be applied to Play Store apps in high-risk categories (§IX).

B. Positive control: detection fires as expected

The synthetic positive-control APK, containing `assets/mobilenet_v1.tflite` (16.9 MB, Google MobileNet v1), is correctly identified by the pipeline:

- Model artifact detected at `assets/mobilenet_v1.tflite`.
- Entropy 7.43 bits/byte, flagged as *structured*, not *high-entropy* (as expected for a stock TFLite model).
- Magic bytes matched; format identified as `tflite`.
- Severity: **High** (“unencrypted model present; architecture and weights readable by anyone who can unzip the APK”).

This establishes that the null result on the F-Droid corpus reflects the absence of ML artifacts in the scanned files, not a tool failure.

C. Per-app severity and Smali coverage

Table II reports per-app severity counts and Smali scan coverage. The coverage column is the most important new datum: the three largest apps had only 11–25% of their Smali files scanned under the default 5 000-file cap. A missed ML-SDK import in the untaken 75–89% would be a silent false negative. Raising the cap is a trivial configuration change for the Play Store pivot and is listed as the top item in §VII.

TABLE II
PER-APP SEVERITY AND SMALI COVERAGE

App	Smali cov. %	Crit	High	Med	Info
aves	75	0	0	0	1
fdroid	25	0	0	0	1
fennec	15	0	0	0	1
newpipe	58	0	0	0	1
nextcloud	11	0	0	0	1
pos. ctrl.	n/a	0	1	0	0

All-Info is the correct outcome when no ML artifacts are present; the tool also surfaces sensitive-permission posture (§VI-D) and records per-APK SHA-256 for reproducibility.

D. Sensitive-permission context

The pipeline reported sensitive-permission posture for every app even where no ML artifacts were present. Fennec declared the most sensitive permissions (6), followed by Nextcloud (5), F-Droid (4), Aves (3), and NewPipe (2). These permissions are *latent* risk: if any of these apps added on-device ML in a future release, the data pipeline would already have access to CAMERA, MIC, FINE_LOCATION, or INTERNET without re-requesting consent.

VII. TOOL LIMITATIONS

- 1) **Smali scan cap.** Nextcloud’s 46 547-file Smali tree was scanned at only 11% coverage; three of five apps fell below 25%. The cap exists to bound pilot runtime and must be raised (or replaced with priority-ordered scanning) before any claim of full coverage.

- 2) **Obfuscation blindness.** ProGuard/R8 renames `org.tensorflow.lite.Interpreter` to, e.g., `a.b.c` before shipping. Pattern-based Smali detection is defeated. This is the dominant risk for the planned Play Store pivot and requires either native-library-anchored heuristics (libraries are rarely obfuscated) or string-constant analysis of the resource table.
- 3) **Entropy is not a cryptographic signal.** High Shannon entropy is consistent with encryption *or* compression; the pipeline labels the condition `high_entropy` (not `likely_encrypted`) and notes the ambiguity in every affected finding.
- 4) **No reflection / dynamic-class-loading analysis.** Smali pattern scan misses ML SDK use hidden behind `Class.forName` or DEX-at-runtime loaders – the same blind spot FlowDroid documents.
- 5) **Static only.** Cannot observe runtime model downloads or federated-learning rounds.
- 6) **Magic-byte signature set is not exhaustive.** Proprietary or custom model formats may evade the set. Entropy is a backstop.

VIII. REPRODUCIBILITY

```
cd cs6491/project
python3 pipeline/pipeline.py \
  --apk-dir apks/ \
  --out results
```

Outputs land in `results/summary.csv` plus one JSON per app. Full decoded trees are written under `decompiled/`. Per-APK SHA-256 is recorded in both JSON and CSV. Runtime was approximately three minutes for the 274 MB corpus on a single Kali Linux laptop (Python 3.13, apktool 2.7.0, jadx 1.5.2).

IX. CONCLUSION AND NEXT STEPS

The pilot validates two claims. First, the pipeline runs end-to-end with tools that ship on standard Linux distributions (apktool, jadx, Python plus defusedxml) – no heroic toolchain is required. Second, the detector fires correctly on a known-positive sample (MobileNet v1) while returning a clean null on a five-app F-Droid corpus.

The next run should target Play Store apps in three high-risk categories: keyboard apps (continuous typing data → retraining surface), translation apps (offline TFLite models), and camera / filter apps (on-device classification on PII). Two pipeline changes are required first: raise the Smali cap (trivial) and add an obfuscation-resistant detector (moderate – anchored on native- library presence plus string-constant extraction).

Paired with a training-set defense that detects Truth Serum poisons (out of scope for this pilot), this auditor completes the loop between lab-side defense research and field-side risk assessment by identifying the real-world training pipelines where such poisons could be planted.

REFERENCES

- [1] F. Tramèr, R. Shokri, A. San Joaquin, H. Le, M. Jagielski, S. Hong, and N. Carlini, “Truth Serum: Poisoning Machine Learning Models to Reveal Their Secrets,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022.
- [2] M. Xu, J. Liu, Y. Liu, F. X. Lin, Y. Liu, and X. Liu, “A First Look at Deep Learning Apps on Smartphones,” in *Proc. World Wide Web Conference (WWW)*, 2019.
- [3] Z. Sun, R. Sun, L. Lu, and A. Mislove, “Mind Your Weight(s): A Large-scale Study on Insufficient Machine Learning Model Protection in Mobile Apps,” in *Proc. USENIX Security Symposium*, 2021.
- [4] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, “FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2014.
- [5] W. Enck, P. Gilbert, B. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, “TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones,” in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010.
- [6] B. Tran, J. Li, and A. Madry, “Spectral Signatures in Backdoor Attacks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [7] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership Inference Attacks Against Machine Learning Models,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2017.
- [8] Google, “ML Kit: On-Device Machine Learning for Mobile,” 2023. [Online]. Available: <https://developers.google.com/ml-kit>